AGT — Shared Services Architecture

Cahier des Charges
Service de Notification Centralisé

Version	1.2
Date	Mars 2026
Statut	Post-audit — Aligné Auth v2.1 + Users v1.0
Classification	Confidentiel

Historique des Révisions

Ver.	Date	Auteur	Description
1.0	2026-03-20	Équipe Architecture	Création initiale du cahier des charges
1.1	2026-03-20	Équipe Architecture	Post-audit : alignement Auth v2.1 + Users v1.0. Identité `user_id` clarifiée. `platform_id` aligné UUID. Contrats inter-services formalisés. Table `device_tokens` ajoutée. MLD corrigé. Préférences : Notification source de vérité.
1.2	2026-03-20	Équipe Architecture	Multi-canal orchestré par Notification (`channels:[]`). `send-bulk` limité à 100. Templates UNIQUE(name, platform_id). Cache coordonnées Users. Vérification user actif. Préférences par défaut. Sécurité device tokens / préférences. Idempotency scoped + TTL. Ordre retry/provider/fallback clarifié. Diagramme PlantUML en annexe.

Contents

1 Introduction **4**

2 Contexte & Problématique **4**
- 2.1 Contexte . . . 4
- 2.2 Problématique . . . 4

3 Vision & Objectifs **4**
- 3.1 Vision . . . 4
- 3.2 Objectifs . . . 5

4 Parties Prenantes & Utilisateurs **5**

5 Besoins Fonctionnels **5**
- 5.1 Envoi de notifications . . . 5
- 5.2 Gestion des templates . . . 6
- 5.3 Préférences utilisateur . . . 6
- 5.4 Notifications in-app . . . 6
- 5.5 Historique & Statistiques . . . 6
- 5.6 Planification . . . 6
- 5.7 Campagnes . . . 7
- 5.8 Device Tokens (v1.1) . . . 7

6 Besoins Non-Fonctionnels **7**

7 Architecture Technique **7**
- 7.1 Vue d'ensemble . . . 7
- 7.2 Stack technique . . . 8

7.3 Canaux supportés & Priorités . . . 8
7.4 Dépendances . . . 8
7.5 Source des coordonnées par canal . . . 9
7.6 Flux d'envoi . . . 9
7.7 Stratégie de fallback providers . . . 10

8 Modélisation **10**
8.1 Diagramme de classes . . . 10
8.2 MLD — Modèle Logique de Données . . . 10
8.2.1 Table `notifications` . . . 10
8.2.2 Table `templates` . . . 11
8.2.3 Table `template_versions` . . . 11
8.2.4 Table `template_variables` . . . 11
8.2.5 Table `user_preferences` . . . 12
8.2.6 Table `notification_logs` . . . 12
8.2.7 Table `campaigns` . . . 12
8.2.8 Table `campaign_recipients` . . . 13
8.2.9 Table `scheduled_notifications` . . . 13
8.2.10 Table `device_tokens` (v1.1) . . . 13
8.2.11 Table `platform_channel_config` (v1.1) . . . 14

9 Contrats d'API **14**
9.1 Envoi . . . 14
9.1.1 POST /notifications/send . . . 14
9.1.2 POST /notifications/send-bulk . . . 15
9.2 Templates . . . 15
9.2.1 POST /templates . . . 15
9.2.2 GET /templates . . . 15
9.2.3 GET /templates/{id} . . . 15
9.2.4 PUT /templates/{id} . . . 16
9.2.5 DELETE /templates/{id} . . . 16
9.2.6 POST /templates/{id}/preview . . . 16
9.2.7 GET /templates/{id}/versions . . . 16
9.3 Préférences . . . 16
9.3.1 GET /users/{userId}/notification-preferences . . . 16
9.3.2 PUT /users/{userId}/notification-preferences . . . 16
9.4 Notifications In-App . . . 17
9.4.1 GET /users/{userId}/notifications . . . 17
9.4.2 GET /users/{userId}/notifications/unread-count . . . 17
9.4.3 PUT /users/{userId}/notifications/{notifId}/read . . . 17
9.4.4 PUT /users/{userId}/notifications/read-all . . . 17
9.4.5 DELETE /users/{userId}/notifications/{notifId} . . . 17
9.5 Planification . . . 17

9.5.1 POST /notifications/schedule . . . 17
9.5.2 GET /notifications/scheduled . . . 18
9.5.3 PUT /notifications/scheduled/{id} . . . 18
9.5.4 DELETE /notifications/scheduled/{id} . . . 18
9.6 Campagnes . . . 18
9.6.1 POST /campaigns . . . 18
9.6.2 GET /campaigns . . . 18
9.6.3 GET /campaigns/{id} . . . 18
9.6.4 GET /campaigns/{id}/progress . . . 18
9.6.5 POST /campaigns/{id}/cancel . . . 18
9.6.6 GET /campaigns/{id}/stats . . . 19
9.7 Device Tokens . . . 19
9.7.1 POST /users/{userId}/device-tokens . . . 19
9.7.2 GET /users/{userId}/device-tokens . . . 19
9.7.3 DELETE /users/{userId}/device-tokens/{tokenId} . . . 19
9.8 Logs & Statistiques . . . 19
9.8.1 GET /notifications/logs . . . 19
9.8.2 GET /notifications/stats . . . 19
9.9 Configuration . . . 20
9.9.1 GET /platforms/{platformId}/channels-priority . . . 20
9.9.2 PUT /platforms/{platformId}/channels-priority . . . 20

10 Contrats Inter-Services **20**
10.1 Auth → Notification . . . 20
10.1.1 Mode de communication . . . 20
10.1.2 Templates pré-requis Auth . . . 21
10.1.3 Comportement en cas d'échec . . . 21
10.2 Users → Notification . . . 21
10.3 Notification → Auth . . . 21
10.4 Notification → Users . . . 21

11 Infrastructure **21**
11.1 Composants d'infrastructure . . . 21
11.2 Services externes à payer . . . 22
11.3 Variables d'environnement . . . 22
11.4 Stratégie d'archivage des logs . . . 22
11.5 Monitoring temps réel . . . 22

Annexes **23**
A. Cas d'usage . . . 23
B. Glossaire . . . 23
C. Code PlantUML — Diagramme de Classes . . . 24

1 Introduction

Ce document constitue le cahier des charges du **Service de Notification Centralisé** destiné à l'écosystème de plateformes AGT. Il sert de référence unique pour la conception, le développement et l'intégration avec les autres services de l'architecture partagée.

Périmètre. Le service couvre l'ensemble du cycle de vie des notifications : envoi multi-canal (email, SMS, push, in-app, WhatsApp), gestion de templates dynamiques, préférences utilisateur, planification, campagnes, historique et statistiques. Il est conçu comme un service centralisé réutilisable par toute plateforme de l'écosystème.

Convention d'identité (v1.1+)

Dans tout ce document :

- `user_id` désigne `users_profiles.id` du Service Users. C'est l'identifiant du profil utilisateur, distinct de l'identifiant d'authentification.
- `platform_id` désigne un UUID provenant du registre des plateformes du Service Auth (`platforms.id`).
- Notification ne stocke aucune table locale de plateformes ni d'utilisateurs. Il consomme les services Auth et Users pour la validation et la résolution des données.

2 Contexte & Problématique

2.1 Contexte

L'organisation développe plusieurs plateformes indépendantes :

Plateforme	Statut	Description
AGT-Bot	En développement	Plateforme de bots automatisés
AGT-Market	En développement	Marketplace de services
SALMA	Migration prévue	Plateforme existante à intégrer
Futures	Planifié	Nouvelles plateformes à venir

Chaque service partagé (Auth, Users, Notification, Média...) est indépendant et accessible via API REST.

2.2 Problématique

Sans service centralisé, chaque plateforme implémente ses propres intégrations d'envoi de notifications, ce qui entraîne : duplication des intégrations avec les providers (SendGrid, Twilio, FCM...), impossibilité de gérer les préférences utilisateur de manière unifiée, absence de traçabilité centralisée, coûts de maintenance élevés et risques d'incohérences.

3 Vision & Objectifs

3.1 Vision

Un point d'entrée unique pour l'envoi de toute notification dans l'écosystème. Aucune plateforme n'envoie directement un email, SMS, push ou WhatsApp. **Tout passe par le Service Notification.**

3.2 Objectifs

#	Objectif	Indicateur
O1	Envoi unifié multi-canal	5 canaux supportés
O2	Templates dynamiques multilingues	CRUD + versionning
O3	Préférences utilisateur respectées	Par canal, catégorie, plateforme
O4	Traçabilité complète	Logs + statuts par notification
O5	Envoi asynchrone performant	Mise en queue $<$ 100ms
O6	Planification et campagnes	Envois différés + masse
O7	Réutilisabilité	Intégration nouvelle plateforme $<$ 1 jour

4 Parties Prenantes & Utilisateurs

Rôle	Type	Responsabilité
Product Owner	Interne	Priorisation des fonctionnalités
Équipe Backend	Interne	Développement et maintenance du service
Équipe Frontend	Interne	Intégration notifications in-app
DevOps	Interne	Infrastructure, CI/CD, monitoring
Services internes	Interne	Consommateurs de l'API (Auth, plateformes)
Utilisateurs finaux	Externe	Réception des notifications, gestion préférences
Administrateurs	Interne	Gestion templates, campagnes, supervision

5 Besoins Fonctionnels

5.1 Envoi de notifications

ID	Fonctionnalité	Priorité
F-SEND-01	Envoi immédiat mono-canal ou multi-canal	Urgent
F-SEND-02	Envoi multi-canal orchestré par Notification avec fallback automatique configurable	Urgent
F-SEND-03	Fallback automatique entre canaux (sauf catégorie `security`)	Urgent
F-SEND-04	Envoi planifié (date/heure future)	Normale
F-SEND-05	Envoi en masse (campagnes) avec throttling	Normale
F-SEND-06	Retry automatique (3 tentatives + backoff exponentiel)	Urgent

Orchestration multi-canal (v1.2)

Le service appelant envoie **un seul appel** avec un champ `channels: ["email", "push"]`. Notification orchestre l'envoi sur chaque canal demandé, applique les préférences utilisateur, et gère le fallback automatique selon la configuration de la plateforme. Le service appelant n'a plus besoin de faire N appels séparés.
Si un seul canal est fourni (`channels: ["sms"]`), le comportement est identique à un envoi mono-canal.

5.2 Gestion des templates

ID	Fonctionnalité	Priorité
F-TPL-01	CRUD complet des templates	Urgent
F-TPL-02	Variables dynamiques (`{{variable}}`)	Urgent
F-TPL-03	Support multilingue (FR, EN minimum)	Urgent
F-TPL-04	Templates par canal et par plateforme	Urgent
F-TPL-05	Versionning des templates	Normale
F-TPL-06	Preview avec variables de test	Normale

5.3 Préférences utilisateur

ID	Fonctionnalité	Priorité
F-PREF-01	Activer/désactiver canaux par utilisateur	Urgent
F-PREF-02	Activer/désactiver catégories (marketing, transactionnel...)	Urgent
F-PREF-03	Préférences par plateforme	Normale
F-PREF-04	Les notifications critiques (OTP, sécurité) ignorent les préférences	Urgent

Source de vérité et valeurs par défaut (v1.2)

Le **Service Notification est la source de vérité** pour les préférences de notification. Si aucune préférence n'existe pour un utilisateur sur une plateforme donnée, les **valeurs par défaut** sont appliquées : tous les canaux activés, catégories `transactional` et `security` activées, catégorie `marketing` désactivée. La catégorie `security` est toujours forcée à `true`.

5.4 Notifications in-app

ID	Fonctionnalité	Priorité
F-INAPP-01	Stockage des notifications in-app	Urgent
F-INAPP-02	Marquer lu / non lu	Urgent
F-INAPP-03	Compteur de non lues (badge)	Urgent
F-INAPP-04	Suppression logique par l'utilisateur	Normale
F-INAPP-05	Marquer toutes comme lues	Normale

5.5 Historique & Statistiques

ID	Fonctionnalité	Priorité
F-LOG-01	Logs complets par notification	Urgent
F-LOG-02	Statuts : pending, sent, delivered, read, failed	Urgent
F-LOG-03	Statistiques agrégées par canal, plateforme, période	Normale

5.6 Planification

ID	Fonctionnalité	Priorité
F-SCHED-01	Créer une notification planifiée	Normale
F-SCHED-02	Modifier / supprimer une planification	Normale
F-SCHED-03	Lister les notifications planifiées	Normale

5.7 Campagnes

ID	Fonctionnalité	Priorité
F-CAMP-01	Création de campagne avec liste de destinataires	Normale
F-CAMP-02	Throttling configurable (N messages/seconde)	Normale
F-CAMP-03	Suivi de progression en temps réel	Normale
F-CAMP-04	Annulation d'une campagne en cours	Normale
F-CAMP-05	Statistiques par campagne	Normale

5.8 Device Tokens (v1.1)

ID	Fonctionnalité	Priorité
F-DEV-01	Enregistrement d'un device token (FCM/APNs) pour un utilisateur	Urgent
F-DEV-02	Suppression d'un device token (déconnexion, désinstallation)	Urgent
F-DEV-03	Liste des device tokens d'un utilisateur	Normale
F-DEV-04	Invalidation de tous les device tokens lors d'une révocation globale de sessions (v1.2)	Urgent

6 Besoins Non-Fonctionnels

ID	Catégorie	Exigence	Cible
NF-01	Performance	Mise en queue d'une notification	$<$ 100ms
NF-02	Performance	Throughput d'envoi	500 msg/s
NF-03	Disponibilité	Uptime du service	99.9%
NF-04	Fiabilité	Retry automatique avec backoff	3 tentatives
NF-05	Sécurité	Authentification par Bearer token (validé via Auth)	Obligatoire
NF-06	Sécurité	Masquage données sensibles dans les logs	Obligatoire
NF-07	Sécurité	Rate limiting par provider	Configurable
NF-08	Scalabilité	Architecture stateless, multi-instance	Horizontal
NF-09	Observabilité	Logs structurés JSON, corrélation par request ID	Obligatoire
NF-10	Traitement	Envoi asynchrone via message broker	Obligatoire

7 Architecture Technique

7.1 Vue d'ensemble

Le Service Notification est un microservice REST stateless, déployé en conteneur Docker. Il reçoit les demandes d'envoi via API, les place en file d'attente (message broker), et des workers asynchrones traitent l'envoi effectif via les providers externes.

7.2 Stack technique

Composant	Technologie	Justification
Langage & Framework	Python 3.11+ / Django REST Framework	Cohérence avec l'écosystème
Base de données	PostgreSQL 15+	Stockage principal
Cache	Redis 7+	Cache coordonnées Users, rate limiting, compteurs badge
Message Broker	RabbitMQ (ou Redis Queue)	File d'attente asynchrone
ORM	Django ORM	Modèles et migrations
Scheduler	Celery + Celery Beat	Tâches planifiées et périodiques
Conteneurisation	Docker + Docker Compose	Déploiement isolé

7.3 Canaux supportés & Priorités

Canal	Priorité par défaut	Provider	Notes
Email	Haute	SendGrid / Mailgun / SES	Fallback entre providers
Push	Haute	FCM / APNs	Nécessite device tokens (table locale)
In-app	Moyenne	Interne (PostgreSQL)	Stockage en base
WhatsApp	Basse	Meta Cloud API	Templates validés par Meta uniquement
SMS	Basse	Twilio / Vonage / Infobip	Critique uniquement (OTP)

Règles de fallback (v1.2)

Ordre de traitement en cas d'échec :

1. **Retry sur le même provider** (3 tentatives, backoff exponentiel : 1s, 2s, 4s).
2. **Changement de provider** sur le même canal (ex: SendGrid → Mailgun → SES).
3. **Fallback inter-canal** vers le canal suivant dans l'ordre de priorité configuré par la plateforme.

Exception : les notifications de catégorie `security` (OTP, 2FA, alertes) **n'autorisent jamais le fallback inter-canal**. Elles sont envoyées exclusivement sur le canal demandé. En cas d'échec définitif sur tous les providers du canal, elles passent en statut `failed`.

7.4 Dépendances

Service	Type	Usage
Service Auth	Interne	Validation des tokens Bearer via `GET /auth/verify-token`. Registre maître des plateformes (`platform_id` = UUID Auth).
Service Users	Interne	Résolution des coordonnées destinataire : email, téléphone (via `GET /users/{id}`). Résultat mis en cache Redis (TTL court).
Email providers	Externe	SendGrid, Mailgun, AWS SES
SMS providers	Externe	Twilio, Vonage, Infobip
Push	Externe	FCM (Android), APNs (iOS)
WhatsApp	Externe	Meta Cloud API
Message broker	Interne	RabbitMQ / Redis Queue

Clarification des dépendances (v1.2)

- **Auth** : validation du Bearer token, vérification que la plateforme est active et que l'utilisateur n'est pas bloqué/désactivé. Notification appelle systématiquement `GET /auth/verify-token`.
- **Users** : résolution de l'email et du téléphone à partir du `user_id`. Les coordonnées sont mises en cache dans Redis (TTL 5 minutes) pour réduire la latence et la charge sur Users.
- **Vérification user actif (v1.2)** : avant tout envoi, le worker vérifie que l'utilisateur est actif (non supprimé, non désactivé). Cette vérification est effectuée soit via le cache des coordonnées Users (champ `status`), soit via `GET /auth/verify-token` si le token est disponible.
- **Préférences de notification** : gérées localement par Notification (table `user_preferences`). Users n'est pas source des préférences.
- **Device tokens push** : gérés localement par Notification (table `device_tokens`). Invalidés en cas de déconnexion globale ou révocation de sessions Auth.

7.5 Source des coordonnées par canal

Canal	Source de la donnée	Champ / Cache
Email	Service Users (cache Redis TTL 5min)	`users_profiles.email`
SMS	Service Users (cache Redis TTL 5min)	`users_profiles.phone`
Push	Notification (local)	`device_tokens.token`
WhatsApp	Service Users (cache Redis TTL 5min)	`users_profiles.phone`
In-app	Aucun provider externe	Stockage local en base

7.6 Flux d'envoi

Flux type : envoi d'une notification (v1.2)

1. Le service appelant envoie `POST /notifications/send` avec Bearer token et `channels: ["email", "push"]`.
2. Notification valide le token via `GET /auth/verify-token` (vérifie signature, expiration, user non bloqué/désactivé, plateforme active, session active).
3. Vérification d'idempotence : si `idempotency_key` est fourni et existe déjà pour cette plateforme (non expiré), retourne 409. Sinon, enregistrement de la clé (TTL 24h).
4. Le `user_id` est résolu : récupération des coordonnées depuis le cache Redis, ou appel `GET /users/{id}` si cache expiré. Vérification que le `status` utilisateur est `active`.
5. Les préférences utilisateur sont vérifiées localement. Si aucune préférence n'existe, les valeurs par défaut sont appliquées. Si la catégorie est `security`, les préférences sont ignorées.
6. Pour chaque canal demandé : le template est résolu par `(name, platform_id)`, la version courante pour la locale est récupérée, le contenu est rendu avec les variables.
7. Une notification est enregistrée en base par canal (statut `pending`) et placée en queue.
8. Un worker asynchrone consomme le message et envoie via le provider approprié.
9. En cas d'échec : (1) retry même provider, (2) changement de provider même canal, (3) fallback inter-canal si autorisé.
10. Le statut est mis à jour : `sent`, `delivered`, `failed`.

7.7 Stratégie de fallback providers

Pour chaque canal, plusieurs providers sont configurés par ordre de priorité :

Canal	Provider 1	Provider 2	Provider 3
Email	SendGrid	Mailgun	AWS SES
SMS	Twilio	Vonage	Infobip
Push	FCM	APNs	—
WhatsApp	Meta Cloud API	—	—

8 Modélisation

8.1 Diagramme de classes

Le diagramme de classes complet est fourni en annexe au format PlantUML (voir Annexe C). Il couvre les entités : Notification, Template, TemplateVersion, TemplateVariable, UserPreference, NotificationLog, Campaign, CampaignRecipient, ScheduledNotification, DeviceToken, PlatformChannelConfig.

8.2 MLD — Modèle Logique de Données

8.2.1 Table `notifications`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
user_id	UUID	NOT NULL	Destinataire (`users_profiles.id`)
platform_id	UUID	NOT NULL	Plateforme source (UUID Auth)
template_id	UUID	FK → templates, NULL	Template utilisé
channel	VARCHAR(20)	NOT NULL	email, sms, push, in_app, whatsapp
category	VARCHAR(30)	NOT NULL	transactional, marketing, security
subject	VARCHAR(255)	NULL	Sujet (email)
body	TEXT	NOT NULL	Contenu rendu
status	VARCHAR(20)	NOT NULL	pending, sent, delivered, read, failed
priority	VARCHAR(10)	DEFAULT 'normal'	low, normal, high, critical
metadata	JSONB	NULL	Données additionnelles
idempotency_key	VARCHAR(100)	NULL	Clé d'idempotence
read_at	TIMESTAMPTZ	NULL	Date de lecture (in-app)
deleted_at	TIMESTAMPTZ	NULL	Suppression logique (in-app)
sent_at	TIMESTAMPTZ	NULL	Date d'envoi effectif
created_at	TIMESTAMPTZ	NOT NULL	Date de création

Contrainte unique : `(platform_id, idempotency_key)` — scoped par plateforme (v1.2). Les clés d'idempotence sont expirées et purgées après 24h via un cron job.

Champs in-app (v1.1+)

Pour les notifications de canal `in_app` :

- `is_read` dans l'API est dérivé de `read_at IS NOT NULL`.
- La suppression par l'utilisateur est un soft delete via `deleted_at`.
- Le statut `read` est mis à jour en parallèle de `read_at` pour cohérence des statistiques.

8.2.2 Table `templates`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
name	VARCHAR(100)	NOT NULL	Nom du template
channel	VARCHAR(20)	NOT NULL	Canal cible
platform_id	UUID	NULL	Plateforme (NULL = global)
category	VARCHAR(30)	NOT NULL	Catégorie de notification
is_active	BOOLEAN	DEFAULT true	Template actif
created_by	UUID	NULL	Créateur
created_at	TIMESTAMPTZ	NOT NULL	Date de création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière modification

Contrainte unique : `(name, platform_id)` (v1.2). Deux plateformes peuvent avoir un template du même nom.

Résolution template_name → template_id (v1.2)

Lorsque l'API reçoit un `template_name` :

1. Lookup par `(name, platform_id)` fourni dans la requête.
2. Si non trouvé avec le `platform_id`, fallback vers `platform_id = NULL` (template global).
3. Vérification `is_active = true`.
4. Récupération de la version courante (`is_current = true`) pour la locale demandée, avec fallback vers locale `fr`.

8.2.3 Table `template_versions`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
template_id	UUID	FK → templates	Template parent
version	INT	NOT NULL	Numéro de version
locale	VARCHAR(5)	NOT NULL	Langue (fr, en)
subject	VARCHAR(255)	NULL	Sujet (email)
body	TEXT	NOT NULL	Corps avec variables {{var}}
is_current	BOOLEAN	DEFAULT false	Version active
created_at	TIMESTAMPTZ	NOT NULL	Date de création

Contrainte unique : `(template_id, version, locale)`.

8.2.4 Table `template_variables`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
template_version_id	UUID	FK → template_versions	Version du template
key	VARCHAR(50)	NOT NULL	Nom de la variable
description	VARCHAR(255)	NULL	Description
required	BOOLEAN	DEFAULT true	Variable obligatoire
default_value	VARCHAR(255)	NULL	Valeur par défaut

8.2.5 Table `user_preferences`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
user_id	UUID	NOT NULL	Utilisateur (`users_profiles.id`)
platform_id	UUID	NOT NULL	Plateforme (UUID Auth)
channel_email	BOOLEAN	DEFAULT true	Email activé
channel_sms	BOOLEAN	DEFAULT true	SMS activé
channel_push	BOOLEAN	DEFAULT true	Push activé
channel_whatsapp	BOOLEAN	DEFAULT true	WhatsApp activé
channel_in_app	BOOLEAN	DEFAULT true	In-app activé
cat_transactional	BOOLEAN	DEFAULT true	Catégorie transactionnelle
cat_marketing	BOOLEAN	DEFAULT false	Catégorie marketing
cat_security	BOOLEAN	DEFAULT true	Catégorie sécurité (toujours forcé à true)
updated_at	TIMESTAMPTZ	NOT NULL	Dernière modification

Contrainte unique : `(user_id, platform_id)`.

8.2.6 Table `notification_logs`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
notification_id	UUID	FK → notifications	Notification associée
channel	VARCHAR(20)	NOT NULL	Canal tenté
provider	VARCHAR(30)	NOT NULL	Provider utilisé
status	VARCHAR(20)	NOT NULL	sent, delivered, failed
attempt	INT	NOT NULL	Numéro de tentative
error_message	TEXT	NULL	Message d'erreur (données sensibles masquées)
provider_message_id	VARCHAR(255)	NULL	ID provider externe
created_at	TIMESTAMPTZ	NOT NULL	Date de tentative

8.2.7 Table `campaigns`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
name	VARCHAR(100)	NOT NULL	Nom de la campagne
platform_id	UUID	NOT NULL	Plateforme source (UUID Auth)
template_id	UUID	FK → templates	Template utilisé
channel	VARCHAR(20)	NOT NULL	Canal d'envoi
status	VARCHAR(20)	NOT NULL	draft, running, completed, cancelled
total_recipients	INT	DEFAULT 0	Nombre total de destinataires
sent_count	INT	DEFAULT 0	Nombre envoyé
failed_count	INT	DEFAULT 0	Nombre en échec
throttle_per_second	INT	DEFAULT 10	Rate limiting
scheduled_at	TIMESTAMPTZ	NULL	Date d'envoi planifiée
started_at	TIMESTAMPTZ	NULL	Début effectif
completed_at	TIMESTAMPTZ	NULL	Fin de campagne
created_by	UUID	NULL	Créateur
created_at	TIMESTAMPTZ	NOT NULL	Date de création

8.2.8 Table `campaign_recipients`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
campaign_id	UUID	FK → campaigns	Campagne associée
user_id	UUID	NOT NULL	Destinataire (`users_profiles.id`)
notification_id	UUID	FK → notifications, NULL	Notification générée
status	VARCHAR(20)	DEFAULT 'pending'	pending, sent, failed, skipped
variables	JSONB	NULL	Variables spécifiques au destinataire
created_at	TIMESTAMPTZ	NOT NULL	Date de création

8.2.9 Table `scheduled_notifications`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
user_id	UUID	NOT NULL	Destinataire (`users_profiles.id`)
platform_id	UUID	NOT NULL	Plateforme source (UUID Auth)
template_id	UUID	FK → templates	Template à utiliser
channel	VARCHAR(20)	NOT NULL	Canal d'envoi
variables	JSONB	NULL	Variables du template
scheduled_at	TIMESTAMPTZ	NOT NULL	Date/heure d'envoi
status	VARCHAR(20)	DEFAULT 'pending'	pending, sent, cancelled
notification_id	UUID	FK → notifications, NULL	Notification créée après envoi
created_at	TIMESTAMPTZ	NOT NULL	Date de création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière modification

8.2.10 Table `device_tokens` (v1.1)

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
user_id	UUID	NOT NULL	Utilisateur (`users_profiles.id`)
platform_id	UUID	NOT NULL	Plateforme (UUID Auth)
token	TEXT	NOT NULL	Token FCM ou APNs
device_type	VARCHAR(20)	NOT NULL	android, ios, web
device_name	VARCHAR(100)	NULL	Nom de l'appareil
is_active	BOOLEAN	DEFAULT true	Token actif
created_at	TIMESTAMPTZ	NOT NULL	Date d'enregistrement
updated_at	TIMESTAMPTZ	NOT NULL	Dernière mise à jour

Contrainte unique : `(user_id, token)`.

Sécurité device tokens (v1.2)

Les device tokens peuvent être invalidés (`is_active = false`) lors d'une déconnexion globale ou d'une révocation de toutes les sessions côté Auth. Lorsque Notification reçoit un signal de blocage ou désactivation d'un utilisateur (via le statut retourné par `verify-token` ou le cache Users), il désactive automatiquement tous les device tokens de cet utilisateur.

8.2.11 Table platform_channel_config (v1.1)

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	UNIQUE, NOT NULL	Plateforme (UUID Auth)
priority_order	JSONB	NOT NULL	Ordre des canaux
fallback_enabled	BOOLEAN	DEFAULT true	Fallback inter-canal activé
created_at	TIMESTAMPTZ	NOT NULL	Date de création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière modification

9 Contrats d'API

Base URL : `/api/v1`. Tous les endpoints retournent du JSON. Tous les endpoints requièrent le header `Authorization: Bearer <token>`, validé via `GET /auth/verify-token`.

9.1 Envoi

9.1.1 POST /notifications/send

Description : Envoi d'une notification mono ou multi-canal à un utilisateur. Notification orchestre l'envoi sur chaque canal demandé.

Request body :

```
{
  "user_id": "uuid (users_profiles.id)",
  "channels": ["email", "push"],
  "template_name": "agt_market_order_confirmed",
  "locale": "fr",
  "variables": {
    "user_name": "Jean Dupont",
    "order_id": "CMD-001",
    "total": "15000 FCFA"
  },
  "priority": "high",
  "category": "transactional",
  "idempotency_key": "uuid-unique (optionnel)"
}
```

Response 202 :

```
{
  "notifications": [
    { "id": "notif-uuid-1", "channel": "email", "status": "pending" },
    { "id": "notif-uuid-2", "channel": "push", "status": "pending" }
  ],
  "message": "Notifications queued"
}
```

Codes : 202 Accepted, 400 Validation error, 401 Unauthorized, 404 Template not found, 409 Idempotency conflict, 429 Rate limited.

Compatibilité mono-canal

Si un seul canal est fourni (`channels: ["sms"]`), le comportement est identique à un envoi mono-canal classique. Le champ `channels` remplace l'ancien champ `channel` (v1.0).

9.1.2 POST /notifications/send-bulk

Description : Envoi en masse à plusieurs utilisateurs. Limité à **100** `user_ids` maximum par appel. Pour des volumes plus importants, utiliser les campagnes.

Request body :

```
{
  "user_ids": ["uuid-1", "uuid-2", "uuid-3"],
  "channels": ["push"],
  "template_name": "promo_weekend",
  "locale": "fr",
  "variables": { "discount": "20%" },
  "category": "marketing"
}
```

Response 202 :

```
{
  "queued_count": 3,
  "message": "Bulk notifications queued"
}
```

Codes : 202 Accepted, 400 Validation error (dont `user_ids` > 100), 401 Unauthorized.

9.2 Templates

9.2.1 POST /templates

Description : Création d'un template. Contrainte UNIQUE sur `(name, platform_id)`.

Request body :

```
{
  "name": "order_confirmed",
  "channel": "email",
  "platform_id": "uuid-or-null",
  "category": "transactional",
  "locale": "fr",
  "subject": "Commande {{order_id}} confirmee",
  "body": "Bonjour {{user_name}}, votre commande..."
}
```

Response 201 :

```
{
  "id": "tpl-uuid",
  "name": "order_confirmed",
  "platform_id": "uuid",
  "version": 1,
  "message": "Template created"
}
```

Codes : 201 Created, 400 Validation error, 409 Name already exists for this platform.

9.2.2 GET /templates

Description : Liste des templates. **Query params :** `?channel=email&platform_id=uuid&page=1&limit=20`

9.2.3 GET /templates/{id}

Description : Détail d'un template avec sa version courante.

9.2.4 PUT /templates/{id}

Description : Mise à jour d'un template. Crée automatiquement une nouvelle version.

9.2.5 DELETE /templates/{id}

Description : Suppression logique (désactivation). **Response 200 :** `{"message": "Template deactivated"}`

9.2.6 POST /templates/{id}/preview

Description : Preview du template avec des variables de test.

Request body :

```
{
  "locale": "fr",
  "variables": {
    "user_name": "Test User",
    "order_id": "CMD-001"
  }
}
```

Response 200 :

```
{
  "subject": "Commande CMD-001 confirmee",
  "body": "Bonjour Test User, votre commande..."
}
```

9.2.7 GET /templates/{id}/versions

Description : Historique des versions d'un template.

9.3 Préférences

9.3.1 GET /users/{userId}/notification-preferences

Description : Lecture des préférences (source de vérité locale). Si aucune préférence n'existe, retourne les valeurs par défaut. **Query params :** `?platform_id=uuid`

Response 200 :

```
{
  "user_id": "uuid (users_profiles.id)",
  "platform_id": "uuid",
  "channels": {
    "email": true, "sms": true,
    "push": true, "whatsapp": false,
    "in_app": true
  },
  "categories": {
    "transactional": true,
    "marketing": false,
    "security": true
  }
}
```

9.3.2 PUT /users/{userId}/notification-preferences

Description : Modification des préférences. **L'utilisateur ne peut modifier que ses propres préférences**, sauf via un token S2S autorisé. La catégorie `security` est ignorée si passée à `false`. **Response 200 :** `{"message": "Preferences updated"}`

9.4 Notifications In-App

9.4.1 GET /users/{userId}/notifications

Description : Liste des notifications in-app (exclut `deleted_at IS NOT NULL`). **Query params :** `?status=unread&page=1&limit=20`

Response 200 :

```
{
  "data": [
    {
      "id": "notif-uuid",
      "subject": "Commande confirmee",
      "body": "Votre commande CMD-001...",
      "is_read": false,
      "category": "transactional",
      "created_at": "2026-03-20T10:00:00Z"
    }
  ],
  "total": 5, "unread_count": 3
}
```

9.4.2 GET /users/{userId}/notifications/unread-count

Response 200 : `{"unread_count": 3}`

9.4.3 PUT /users/{userId}/notifications/{notifId}/read

Response 200 : `{"message": "Notification marked as read"}`

9.4.4 PUT /users/{userId}/notifications/read-all

Response 200 : `{"message": "All notifications marked as read", "updated_count": 3}`

9.4.5 DELETE /users/{userId}/notifications/{notifId}

Description : Suppression logique. **Response 200 :** `{"message": "Notification deleted"}`

9.5 Planification

9.5.1 POST /notifications/schedule

Description : Planifier une notification pour envoi futur.

Request body :

```
{
  "user_id": "uuid (users_profiles.id)",
  "channels": ["email"],
  "template_name": "weekly_digest",
  "locale": "fr",
  "variables": { "week": "12" },
  "scheduled_at": "2026-03-25T08:00:00Z"
}
```

Response 201 :

```
{
  "id": "sched-uuid",
  "status": "pending",
  "scheduled_at": "2026-03-25T08:00:00Z",
  "message": "Notification scheduled"
}
```

9.5.2 GET /notifications/scheduled

Query params : `?status=pending&page=1`

9.5.3 PUT /notifications/scheduled/{id}

Description : Modifier la date ou les paramètres.

9.5.4 DELETE /notifications/scheduled/{id}

Response 200 : `{"message": "Scheduled notification cancelled"}`

9.6 Campagnes

9.6.1 POST /campaigns

Request body :

```
{
  "name": "Promo Ete 2026",
  "platform_id": "uuid",
  "template_name": "promo_summer",
  "channel": "email",
  "locale": "fr",
  "user_ids": ["uuid-1", "uuid-2"],
  "throttle_per_second": 50,
  "scheduled_at": "2026-06-01T09:00:00Z"
}
```

Response 201 :

```
{
  "id": "camp-uuid",
  "name": "Promo Ete 2026",
  "status": "draft",
  "total_recipients": 2,
  "message": "Campaign created"
}
```

9.6.2 GET /campaigns

Query params : `?status=running&page=1&limit=20`

9.6.3 GET /campaigns/{id}

Description : Détail d'une campagne.

9.6.4 GET /campaigns/{id}/progress

Response 200 :

```
{
  "total": 5000,
  "sent": 3200,
  "failed": 15,
  "pending": 1785,
  "progress_percent": 64.3
}
```

9.6.5 POST /campaigns/{id}/cancel

Response 200 : `{"message": "Campaign cancelled", "sent_before_cancel": 3200}`

9.6.6 GET /campaigns/{id}/stats

Response 200 :

```
{
  "total_recipients": 5000,
  "sent": 4985,
  "delivered": 4900,
  "failed": 15,
  "read": 2100,
  "delivery_rate": 98.3,
  "read_rate": 42.0
}
```

9.7 Device Tokens

9.7.1 POST /users/{userId}/device-tokens

Request body :

```
{
  "token": "fcm-token-xxx...",
  "device_type": "android",
  "device_name": "Samsung Galaxy S24",
  "platform_id": "uuid"
}
```

Response 201 : `{"id": "dt-uuid", "message": "Device token registered"}`

9.7.2 GET /users/{userId}/device-tokens

Description : Liste des device tokens.

9.7.3 DELETE /users/{userId}/device-tokens/{tokenId}

Response 200 : `{"message": "Device token removed"}`

9.8 Logs & Statistiques

9.8.1 GET /notifications/logs

Query params : `?channel=email&status=failed&from=2026-03-01&to=2026-03-20&page=1`

Response 200 :

```
{
  "data": [
    {
      "id": "log-uuid",
      "notification_id": "notif-uuid",
      "channel": "email",
      "provider": "sendgrid",
      "status": "delivered",
      "attempt": 1,
      "created_at": "2026-03-20T10:00:05Z"
    }
  ],
  "total": 150, "page": 1
}
```

9.8.2 GET /notifications/stats

Query params : `?platform_id=uuid&period=last_30d`

Response 200 :

```
{
  "total_sent": 125000,
  "by_channel": {
    "email": 80000, "push": 30000,
    "sms": 5000, "in_app": 8000,
    "whatsapp": 2000
  },
  "by_status": {
    "delivered": 120000, "failed": 3000,
    "pending": 2000
  },
  "delivery_rate": 96.0
}
```

9.9 Configuration

9.9.1 GET /platforms/{platformId}/channels-priority

Response 200 :

```
{
  "platform_id": "uuid",
  "priority_order": ["email","push","in_app","whatsapp","sms"],
  "fallback_enabled": true
}
```

9.9.2 PUT /platforms/{platformId}/channels-priority

Request body : Même structure. **Response 200 :** `{"message": "Channel priority updated"}`

10 Contrats Inter-Services

10.1 Auth → Notification

Le Service Auth appelle Notification pour l'envoi de notifications de sécurité (OTP, vérification, reset, magic link).

10.1.1 Mode de communication

Paramètre	Valeur
Transport	HTTP POST synchrone via `POST /api/v1/notifications/send`
Auth	Token S2S (Auth s'authentifie via `POST /auth/s2s/token`)
Catégorie	Toujours `security`
Priorité	Toujours `high` ou `critical`
Fallback inter-canal	Interdit (catégorie `security`)
Idempotency	Champ `idempotency_key` fourni par Auth (scoped par platform_id, TTL 24h)

10.1.2 Templates pré-requis Auth

Template name	Canal	Déclencheur Auth	Variables attendues
auth_verify_email	email	POST /auth/register (email)	verification_url, expires_in_minutes, platform_name
auth_otp_sms	sms	register (phone), login/phone	otp_code, expires_in_minutes, platform_name
auth_reset_password	email	POST /auth/forgot-password	reset_url, expires_in_minutes, platform_name
auth_magic_link	email	POST /auth/login/magic-link	magic_link_url, expires_in_minutes, platform_name

10.1.3 Comportement en cas d'échec

L'échec de Notification ne bloque **jamais** une opération Auth. Auth crée l'utilisateur/le token quand même et l'utilisateur peut redemander l'envoi.

10.2 Users → Notification

Événement	Canal	Catégorie	Déclencheur Users
role_assigned	email + in_app	transactional	POST /users/{id}/roles
role_removed	email + in_app	transactional	DELETE /users/{id}/roles/{roleId}
document_validated	email + in_app	transactional	PUT .../status (validated)
document_rejected	email + in_app	transactional	PUT .../status (rejected)

Templates pré-requis : `users_role_assigned`, `users_role_removed`, `users_document_validated`, `users_document_rejected` (avec `platform_id = NULL` pour templates globaux).

10.3 Notification → Auth

Validation Bearer token via `GET /auth/verify-token` à chaque requête API entrante. Vérifie : signature JWT, expiration, user non bloqué/désactivé, plateforme active, session active.

10.4 Notification → Users

Résolution des coordonnées destinataire via `GET /users/{id}`. Résultat mis en cache Redis (TTL 5 min). Vérification que le statut utilisateur est `active` avant envoi.

11 Infrastructure

11.1 Composants d'infrastructure

Service	Technologie	Rôle
Backend API	Python 3.11+ / Django REST Framework	API REST principale
Workers asynchrones	Celery + RabbitMQ	Traitement des envois
Scheduler	Celery Beat	Envois planifiés, purge idempotency keys, purge device tokens inactifs
Base de données	PostgreSQL 15+	Stockage principal
Cache	Redis 7+	Cache coordonnées Users, rate limiting, compteurs badge, idempotency
Conteneur	Docker + Docker Compose	Déploiement isolé
Reverse Proxy	Nginx / Traefik	TLS termination, load balancing

11.2 Services externes à payer

Catégorie	Fournisseurs	Facturation
Email	SendGrid, Mailgun, AWS SES	Par volume
SMS	Twilio, Vonage, Infobip	Par SMS
Push	FCM (gratuit), APNs (gratuit)	Gratuit
WhatsApp	Meta Cloud API	Par message
Hébergement	AWS, GCP, DigitalOcean	Conteneurs, BDD, Redis

11.3 Variables d'environnement

Variable	Description
DATABASE_URL	URL de connexion PostgreSQL
REDIS_URL	URL de connexion Redis
AUTH_SERVICE_URL	URL du Service Auth
USERS_SERVICE_URL	URL du Service Users
RABBITMQ_URL	URL du message broker
SENDGRID_API_KEY	Clé API SendGrid
TWILIO_ACCOUNT_SID	Identifiant Twilio
TWILIO_AUTH_TOKEN	Token Twilio
FCM_SERVER_KEY	Clé serveur Firebase
APNS_KEY_ID	Identifiant clé APNs
META_WHATSAPP_TOKEN	Token Meta Cloud API
SECRET_KEY	Clé secrète Django
USER_CACHE_TTL	TTL cache coordonnées Users en secondes (défaut : 300)
IDEMPOTENCY_TTL	TTL des clés d'idempotence en secondes (défaut : 86400)

11.4 Stratégie d'archivage des logs

Période	Stockage	Action
0–90 jours	PostgreSQL (`notification_logs`)	Accès temps réel
90–365 jours	Table partitionnée / archivée	Requêtes analytiques
> 365 jours	Stockage froid (S3 / GCS)	Conservation légale, suppression PII

11.5 Monitoring temps réel

Dashboard de monitoring

Métriques clés :

- Taux de livraison par canal et par provider (cible > 96%)
- Latence de mise en queue (cible < 100ms P95)
- File d'attente : taille et temps de traitement moyen
- Taux d'erreur par provider (alertes si > 5%)
- Campagnes actives : progression temps réel
- Rate limiting : consommation par provider
- Cache hit ratio coordonnées Users (v1.2)

Annexes

A. Cas d'usage

CU-01 : OTP via Service Auth. Le Service Auth appelle `POST /notifications/send` avec `user_id` (`users_profiles.id`), `channels: ["sms"]`, `template_name: "auth_otp_sms"`, `category: "security"`, `priority: "critical"`, `idempotency_key: "uuid-unique"`. Notification ignore les préférences (catégorie security), résout le numéro via Users (ou cache), et envoie. Aucun fallback inter-canal.

CU-02 : Confirmation de commande (AGT-Market). AGT-Market envoie **un seul appel** `POST /notifications/send` avec `channels: ["email", "push"]`, `template_name: "order_confirmed"`, `category: "transactional"`. Notification orchestre l'envoi sur les deux canaux, résout les templates par `(name, platform_id)`, et applique les préférences utilisateur.

CU-03 : Campagne marketing. Un administrateur crée une campagne via `POST /campaigns` avec 10 000 destinataires et un throttle de 50 msg/s. L'admin suit la progression via `GET /campaigns/{id}/progress`.

CU-04 : Désactivation des préférences. Un utilisateur désactive les notifications marketing sur AGT-Bot via `PUT /users/{userId}/notification-preferences`. Seul l'utilisateur propriétaire (ou un token S2S) peut modifier ses préférences.

CU-05 : Dashboard admin. Un administrateur consulte `GET /notifications/stats` pour les taux de livraison et erreurs par canal.

CU-06 : Enregistrement device token push. L'app mobile appelle `POST /users/{userId}/device-tokens` avec le token FCM. Lors d'un envoi push, le worker récupère tous les device tokens actifs.

CU-07 : Notification de changement de rôle. Users appelle `POST /notifications/send` avec `channels: ["email", "in_app"]`, `template_name: "users_role_assigned"`, `category: "transactional"`.

CU-08 : Utilisateur désactivé (v1.2). Un utilisateur est désactivé côté Auth. Au prochain envoi, le worker détecte via le cache Users ou `verify-token` que l'utilisateur est inactif. L'envoi est annulé (statut `failed`, raison `user_inactive`). Les device tokens sont désactivés.

B. Glossaire

Terme	Définition
Fallback	Mécanisme de repli automatique vers un canal ou provider alternatif. Interdit pour `security`.
Throttling	Limitation du débit d'envoi pour respecter les quotas des providers.
Backoff exponentiel	Stratégie de retry où le délai augmente exponentiellement (1s, 2s, 4s).
Template	Modèle de notification avec des variables dynamiques (`{{var}}`).
In-app	Notification affichée dans l'interface de la plateforme.
Worker	Processus asynchrone consommant les messages de la file d'attente.
Rate limiting	Limitation du nombre de requêtes par unité de temps.
Provider	Service tiers d'envoi (SendGrid, Twilio, FCM, Meta Cloud API...).
Celery	Framework Python de traitement de tâches asynchrones distribuées.
Bearer token	Jeton d'authentification validé via `GET /auth/verify-token`.
Device token	Identifiant unique attribué par FCM/APNs à un appareil pour les notifications push.
Idempotency key	Clé unique scoped par `(platform_id, key)` avec TTL 24h pour éviter les doublons.
`user_id`	`users_profiles.id` du Service Users.
`platform_id`	UUID de la plateforme, registre Auth.
S2S	Service-to-Service — communication authentifiée via `POST /auth/s2s/token`.

C. Code PlantUML — Diagramme de Classes

Le code PlantUML ci-dessous correspond au diagramme de classes complet du Service Notification v1.2. Il peut être rendu sur `plantuml.com` ou tout outil compatible.

```
@startuml Notification Service - Class Diagram v1.2

skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "notifications" as notif {
  * id : UUID <<PK>>
  --
  * user_id : UUID
  * platform_id : UUID
  template_id : UUID <<FK>>
  * channel : VARCHAR(20)
  * category : VARCHAR(30)
  subject : VARCHAR(255)
  * body : TEXT
  * status : VARCHAR(20)
  priority : VARCHAR(10) = 'normal'
  metadata : JSONB
  idempotency_key : VARCHAR(100)
  read_at : TIMESTAMPTZ
  deleted_at : TIMESTAMPTZ
  sent_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  ..
  <<UQ>> (platform_id, idempotency_key)
}

entity "templates" as tpl {
  * id : UUID <<PK>>
  --
  * name : VARCHAR(100)
  * channel : VARCHAR(20)
  platform_id : UUID
  * category : VARCHAR(30)
  is_active : BOOLEAN = true
  created_by : UUID
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (name, platform_id)
}

entity "template_versions" as tplv {
  * id : UUID <<PK>>
  --
  * template_id : UUID <<FK>>
  * version : INT
  * locale : VARCHAR(5)
  subject : VARCHAR(255)
  * body : TEXT
  is_current : BOOLEAN = false
  * created_at : TIMESTAMPTZ
  ..
  <<UQ>> (template_id, version, locale)
}

entity "template_variables" as tplvar {
  * id : UUID <<PK>>
  --
  * template_version_id : UUID <<FK>>
  * key : VARCHAR(50)
  description : VARCHAR(255)
  required : BOOLEAN = true
  default_value : VARCHAR(255)
}

entity "user_preferences" as pref {
  * id : UUID <<PK>>
  --
  * user_id : UUID
  * platform_id : UUID
  channel_email : BOOLEAN = true
```

```
  channel_sms : BOOLEAN = true
  channel_push : BOOLEAN = true
  channel_whatsapp : BOOLEAN = true
  channel_in_app : BOOLEAN = true
  cat_transactional : BOOLEAN = true
  cat_marketing : BOOLEAN = false
  cat_security : BOOLEAN = true
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (user_id, platform_id)
}

entity "notification_logs" as logs {
  * id : UUID <<PK>>
  --
  * notification_id : UUID <<FK>>
  * channel : VARCHAR(20)
  * provider : VARCHAR(30)
  * status : VARCHAR(20)
  * attempt : INT
  error_message : TEXT
  provider_message_id : VARCHAR(255)
  * created_at : TIMESTAMPTZ
}

entity "campaigns" as camp {
  * id : UUID <<PK>>
  --
  * name : VARCHAR(100)
  * platform_id : UUID
  * template_id : UUID <<FK>>
  * channel : VARCHAR(20)
  * status : VARCHAR(20)
  total_recipients : INT = 0
  sent_count : INT = 0
  failed_count : INT = 0
  throttle_per_second : INT = 10
  scheduled_at : TIMESTAMPTZ
  started_at : TIMESTAMPTZ
  completed_at : TIMESTAMPTZ
  created_by : UUID
  * created_at : TIMESTAMPTZ
}

entity "campaign_recipients" as camprec {
  * id : UUID <<PK>>
  --
  * campaign_id : UUID <<FK>>
  * user_id : UUID
  notification_id : UUID <<FK>>
  status : VARCHAR(20) = 'pending'
  variables : JSONB
  * created_at : TIMESTAMPTZ
}

entity "scheduled_notifications" as sched {
  * id : UUID <<PK>>
  --
  * user_id : UUID
  * platform_id : UUID
  * template_id : UUID <<FK>>
  * channel : VARCHAR(20)
  variables : JSONB
  * scheduled_at : TIMESTAMPTZ
  status : VARCHAR(20) = 'pending'
  notification_id : UUID <<FK>>
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

entity "device_tokens" as devtok {
  * id : UUID <<PK>>
  --
  * user_id : UUID
  * platform_id : UUID
  * token : TEXT
  * device_type : VARCHAR(20)
  device_name : VARCHAR(100)
```

```
  is_active : BOOLEAN = true
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (user_id, token)
}

entity "platform_channel_config" as pcc {
  * id : UUID <<PK>>
  --
  * platform_id : UUID <<UQ>>
  * priority_order : JSONB
  fallback_enabled : BOOLEAN = true
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

tpl ||--o{ tplv : "1..* versions"
tplv ||--o{ tplvar : "0..* variables"
tpl ||--o{ notif : "0..* notifications"
tpl ||--o{ camp : "0..* campaigns"
tpl ||--o{ sched : "0..* scheduled"
notif ||--o{ logs : "1..* logs"
camp ||--o{ camprec : "1..* recipients"
camprec }o--|| notif : "0..1 notification"
sched }o--o| notif : "0..1 notification"

@enduml
```

Fin du document — AGT Engineering — v1.2